

Testabdeckung

Code Coverage

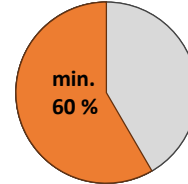
- Anteil des Codes, der **quantitativ** durch Tests abgedeckt wird
- Ziel ist es sicherzustellen, dass der größte Teil des Codes getestet ist, um potentielle Fehler frühzeitig zu erkennen
- Werkzeuge
 - In Visual Studio Ultimate/Enterprise integriert
 - [Fine Code Coverage](#) Visual Studio Extension
 - [Ncover](#) (kostenpflichtig)
 - [Coverlet](#) für .NET Core und .NET 5+

Arten des Code Coverage

- | | |
|----------------------|-----------------------------------|
| • Statement Coverage | Jede Anweisung min. 1x |
| • Branch Coverage | Jede mögliche Verzweigung min. 1x |
| • Path Coverage | Jeder mögliche Pfad min. 1x |
| • Function Coverage | Jede Funktion min. 1x |

Best Practices

- Regelmäßig überprüfen und sicherstellen, dass neue Codeänderungen abgedeckt sind. (PR Constraints in CI-Prozess)
- Sollte in Kombination mit Unit Tests, Integrationstests und anderen Testmethoden verwendet werden.
- Coverage
 - 100% unrealistisch und nicht immer notwendig.
 - Ziel sollte sein, kritische Teile des Codes abzudecken.



Live Unit Testing

- Unit Tests werden im Hintergrund permanent ausgeführt
- reagieren auf Code Änderungen
- ermöglicht Echtzeit Feedback
- Frühzeitige Fehlererkennung kann Produktivität erhöhen

Aktuell noch probleme z.b. bei data driven tests in mstest und auch mit typemock isolator

Live Unit Testing mit NCrunch

- ncrunch.net (159-289\$)
- Bietet zusätzliche Metriken, Berichte, Performance Profiler
- Parallelisierung und CI/CD-Integration
- History der Testdaten
- Visualisierung ob Codezeilen „grün/rot“ oder noch nicht getestet sind

Aktuell noch probleme z.b. bei data driven tests in mstest und auch mit typemoock isolator

```
....._citiesByPostalCode.Add(postalCode, city);  
.....return city;  
.....catch (WebException ex)  
.....{  
....._logger.Write("Error 1002: " + ex.Message);  
.....}
```

Was ist Mutation Testing?

- Fortgeschrittene Testmethode, die den Code verändert (mutiert) und überprüft, ob die Tests diese Änderungen erkennen.
- Ziel soll es sein sicherzustellen, dass die Tests robust genug sind, um Fehler im Code zu erkennen.
- Wenn Tests nicht fehlschlagen, könnte dies darauf hindeuten, dass Tests den Code nicht ausreichend abdecken.

Was sind Mutationstests?

Qualität statt Quantität
<https://www.youtube.com/watch?v=sGwfwtkADfk>

Ziel:

~~Geschwindigkeit: Je nach Codebase kann es dauern~~

7

But wait, what about code coverage?

Well... code coverage doesn't tell you everything about the effectiveness of your tests.

Think about it, when was the last time you saw a test without an assertion, purely to increase the code coverage?

Imagine a sandwich covered with paste.

Code coverage would tell you the bread is 80% covered with paste.

Mutation testing, on the other hand, would tell you it is actually *chocolate* paste and not... well... something else.

Mutation Testing - Stryker

- Stryker.NET CLI installieren

```
dotnet tool install -g dotnet-stryker
```
- Projekt aufsetzen

```
dotnet new tool-manifest
dotnet tool install dotnet-stryker
dotnet tool restore
```



- Stryker ausführen

```
dotnet stryker
```

Was sind Mutationstests?

<https://www.youtube.com/watch?v=sGwfwtkADfk>

<https://stryker-mutator.io>

[Getting Started \(.NET\)](#)

8

Geschwindigkeit: Je nach Codebase kann es dauern

But wait, what about code coverage?

Well... code coverage doesn't tell you everything about the effectiveness of your tests.

Think about it, when was the last time you saw a test without an assertion, purely to increase the code coverage?

Imagine a sandwich covered with paste.

Code coverage would tell you the bread is 80% covered with paste.

Mutation testing, on the other hand, would tell you it is actually *chocolate* paste and not... well... something else.

Stryker Konfiguration

- stryker-config.json im Stammverzeichnis des Projekts

- Stryker ausführen

dotnet stryker

- Ergebnisse im **StrykerOutput** Verzeichnis als HTML-Bericht

```
{  
  "mode": "full",  
  "reporters": ["cleartext", "html", "dashboard"],  
  "dashboardApiKey": "YOUR_API_KEY",  
  "dashboardProject": "YOUR_PROJECT_NAME",  
  "dashboardVersion": "YOUR_VERSION",  
  "mutate": ["**/*.cs"],  
  "timeoutMs": 30000,  
  "thresholds": {  
    "high": 80,  
    "low": 60,  
    "break": 50  
  }  
}
```

All files Stryker.NET Report



Mutants

Tests

All files

Q



File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	54.35	80.65	25	6	0	15	13	0	3	25	21	62
Refactored	46.88	71.43	15	6	0	11	9	0	3	15	17	44
Vorgabe/Program.cs	71.43	100.00	10	0	0	4	4	0	0	10	4	18

Codequalität

Was genau ist Codequalität?

- Funktionalität
 - Konzentration auf Anforderungen
 - Tests
- Zuverlässigkeit
 - Exception Handling
- Bedienbarkeit
 - Corporate UI Design, optimale Oberflächenführung
- Effizienz
 - performant, skalierbar, speicherschonend

Was genau ist Codequalität?

- Wartbarkeit
 - Standardisierte Entwicklung
 - Software Design
 - Lesbarer Code
- Austauschbarkeit
 - Entkopplung
 - Schnittstellendesign plattformneutral/technologieneutral
 - Komponentenaufteilung

Programmierstile

- **Prozedurale Entwicklung**
 - Einfaches „runterprogrammieren“
 - Vermischung der Schichten
 - Wiederverwendbarkeit: schwer bis gar nicht
- **Objektorientierte Entwicklung (OOP)**
 - Entwicklung auf Basis von OO-Grundsätzen (Patterns)
 - Setzt Design voraus (OOD)
 - Wiederverwendbarkeit: besser

Entwicklungsgrundsätze

- YAGNI: You ain't gonna need it
- ~~DRY: Don't repeat yourself~~ Gilt nicht für Tests
- „Premature Optimization is the root of all evil“
 - Performanceoptimierung auf Basis von Analyse und Fakten
 - Performanceoptimierter Code oft schlechter lesbar
 - Geschäftsanwendung vs. Echtzeit-FPS?
 - Beispiel "[Fast inverse square root](#)" von id-Software

$$\hat{v} = v \frac{1}{\sqrt{v_1^2 + v_2^2 + v_3^2}}$$



Code Analyse

Visual Studio Code Analysis

- In Visual Studio integriert
- Indikatoren für Wartbarkeit und Testaufwand
- Potenzielle Fehler & Sicherheitslücken identifizieren
- Schwellenwerte für Analyseregeln anpassbar

Code Metrics Results

Filter: None Min: Max:

Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling	Lines of Code
BusinessLayer (Release)	38	545	1	9	565
BusinessLayer	38	545	1	9	565
Address	37	265	1	7	275
Address(int, string, string)	76	1	0	0	4
Id.get() : int	98	1	0	0	1
LoadAddress(int) : Address	18	102	7	108	
Save() : void	7	159	3	160	
StreetAddress1.get() : string	98	1	0	1	
StreetAddress2.get() : string	98	1	0	1	
Customer	38	280	1	7	290
Address.get() : Address	98	1	1	1	
Customer(int, string, string)	76	1	0	4	
FirstName.get() : string	98	1	0	1	
Id.get() : int	98	1	0	1	
LastName.get() : string	98	1	0	1	
LoadCustomer(int) : Customer	8	146	6	152	
Save() : void	13	129	2	130	
DataAccessLayer (Release)	95	6	1	2	6
MainApplication (Release)	84	10	7	5	16

<http://www.bryancook.net/2011/10/guided-by-testswrap-up.html>

Zyklomatische Komplexität

- Maß für die Komplexität von Programmcode
- Misst Anzahl unabhängiger Pfade in einer Methode
- Basiert auf Verzweigungen und Bedingungen
- Bewertung
 - 1-10: geringe Komplexität
 - 11-20: mittlere Komplexität
 - 21-50: hohe Komplexität

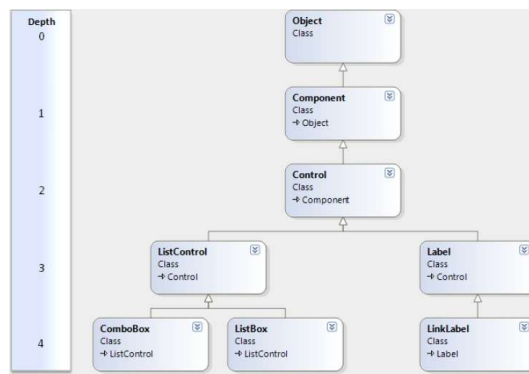
Complexity	
1	<code>bool ParseCommandLine(string[] arguments)</code>
2	<code>{</code>
	<code>if (arguments.Length == 0)</code>
	<code>{</code>
	<code> ShowHelp();</code>
	<code> return false;</code>
	<code>}</code>
3	<code>for (int i = 0; i < arguments.Length; i++)</code>
4	<code>{</code>
	<code> if (arguments[i] == "/?")</code>
	<code> {</code>
	<code> ShowHelp();</code>
	<code> return false;</code>
	<code> }</code>
5	<code> if (arguments[i] == "/input")</code>
6,7	<code> {</code>
	<code> if (arguments.Length > 1 && File.Ex</code>
	<code> {</code>
	<code> InputFileName = arguments[i++];</code>
	<code> }</code>
	<code> }</code>
	<code>return true;</code>
	<code>}</code>

Vererbungstiefe

- Länge der Hierarchie in OOP-Code
- Indikator für Komplexität
- Hilft bei der Identifizierung von Risiken in der Softwarearchitektur
- Indikator für Wartbarkeit und Testaufwand

Behandlung von Verstößen

- Favor composition over inheritance
- Hierarchieebenen mittels Interfaces und Delegation reduzieren

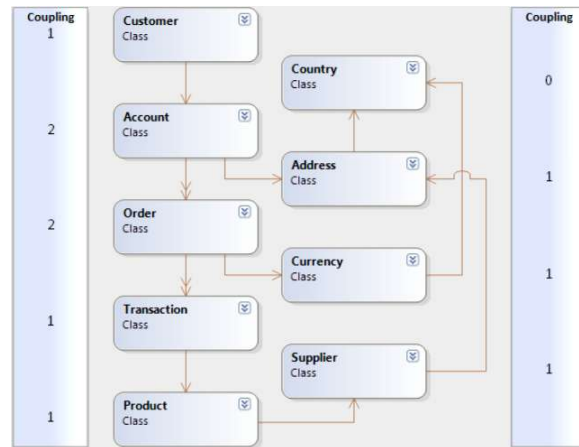


Klassenkopplung

- Misst Kopplung durch Zählen eindeutiger Typverweise
- Warnt vor übermäßiger Kopplung, die die Wartbarkeit erschwert
- Fördert geringere Kopplung und höhere Kohäsion

Behandlung von Verstößen

- Umgestaltung des Typs oder der Methode zur Reduzierung der Kopplungen



Anzahl Quellcodezeilen

Vorteile

- Leicht verständlich und messbar
- Ermöglicht schnelle Projektgrößenvergleiche
- Basis für Aufwandsschätzungen

Nachteile

- Berücksichtigt nicht Codequalität oder Komplexität
- Kann durch Programmierstil beeinflusst werden
- Nicht immer aussagekräftig für Produktivität oder Effizienz

```
Lines 111 <summary>
      112     Parses the specified command-line arguments.
      113 </summary>
      114 bool ParseCommandLine(string[] arguments)
      115 {
      116     // By default we show help if no
      117     // command-line arguments are specified
      118     if (arguments.Length == 0)
      119     {
      120         ShowHelp();
      121         return false;
      122     }
      123
      124     for (int i = 0; i < arguments.Length; i++)
      125     {
      126         if (arguments[i] == "/?")
      127         {
      128             ShowHelp();
      129             return false;
      130         }
      131
      132         if (arguments[i] == "/input")
      133         {
      134             // We only recognize the /input
      135             // switch, if the file name exists
      136             if (arguments.Length > 1 && File.Exists(arguments[i + 1]))
      137             {
      138                 InputFileName = arguments[i++];
      139             }
      140         }
      141     }
      142
      143     return true;
      144 }
```

Wartbarkeitsindex

- Zusammengesetzte Metrik
- Berechnung basiert auf
 - Halstead-Volumen
 - Zyklomatische Komplexität
 - Anzahl der Codezeilen
- Hilft bei schneller Identifizierung problematischer Codebereiche
- Unterstützt Entwicklungsteams bei der Priorisierung von Refactoring-Maßnahmen

Icon	Level	Range
■	High Maintainability	Between 20 and 100 inclusive
▲	Moderate Maintainability	Between 10 and 19 inclusive
●	Low Maintainability	Between 0 and 9 inclusive

F (Komplexität,
Vererbungstiefe,
Klassenkopplung,
Anzahl Codezeilen)

Programmierrichtlinien

```
/// <summary>
/// Prüft, ob ein Prozess mit dem übergebenen Prozessnamen läuft.
/// </summary>
/// <param name="processName">Prozessname</param>
/// <returns>true, falls Prozess läuft. Sonst false.</returns>
public static bool IsProcessRunning(string processName)
{
    Process[] runningProcesses = Process.GetProcessesByName(processName);
    return runningProcesses != null && runningProcesses.Length > 0;
}
```

Defensive Programmierung

- Eine Programmierpraxis, um Software robust und fehlertolerant zu gestalten.
- Fokus: potenzielle Fehler und unerwartete Eingaben frühzeitig erkennen und behandeln
- Stabilität und Zuverlässigkeit des Systems gewährleisten

https://de.wikipedia.org/wiki/Defensives_Programmieren

Defensive Programmierung ist eine Programmierpraxis, die darauf abzielt, Software robust und fehlertolerant zu gestalten. Der Fokus liegt darauf, potenzielle Fehler und unerwartete Eingaben frühzeitig zu erkennen und zu behandeln, um die Stabilität und Zuverlässigkeit des Systems zu gewährleisten. Hier sind die wesentlichen Merkmale der defensiven Programmierung und eine Bewertung, ob die auf der Folie genannten Punkte tatsächlich darauf zutreffen:

Best Practices Defensiver Programmierung

- Selbstdokumentierender Code und Kommentare
 - Lesbarer Code => wartbarer Code
- Fehlerbehandlung und Validierung von Eingaben
 - Fange Ausnahmen nur dort ab, wo Fehler kompensiert werden können
 - Logging und Überwachung von Ausnahmen
- Kapselung und Zugriffskontrolle
 - Entkopplung fördern mittels Schnittstellen und Abstraktionen

- Selbstdokumentierender Code
 - Code sollte so geschrieben sein, dass er leicht verständlich und nachvollziehbar ist.
 - Verwende aussagekräftige Variablen- und Methodennamen.
- Namenskonventionen
 - Halte dich an konsistente Namenskonventionen, um die Lesbarkeit und Wartbarkeit des Codes zu verbessern.
 - Beispiele: CamelCase für Methoden, PascalCase für Klassen.
- Kommentare
 - Verwende Kommentare, um komplexe Abschnitte des Codes zu erklären.
 - Achte darauf, dass Kommentare aktuell und relevant bleiben.
- Zugriffsmodifizier
 - Nutze Zugriffsmodifizierer (wie private, protected, public), um den Zugriff auf Klassenmitglieder zu kontrollieren.
 - Minimiere die Sichtbarkeit von Klassenmitgliedern, um die Kapselung zu gewährleisten.
- Schnittstellen
 - Verwende Schnittstellen, um Abhängigkeiten zu reduzieren und den Code flexibler zu gestalten.
 - Fördert die Entkopplung und erleichtert das Testen und die Wartung.

- Validierungen und Überprüfungen
 - Führe Validierungen und Überprüfungen durch, um sicherzustellen, dass Eingaben und Zustände gültig sind.
 - Verwende Assertionen und Ausnahmen, um ungültige Zustände zu erkennen und zu behandeln.
- Fehlerbehandlung
 - Implementiere eine robuste Fehlerbehandlung, um unerwartete Fehler zu erkennen und angemessen darauf zu reagieren.
 - Verwende Try-Catch-Blöcke und benutzerdefinierte Ausnahmen.
- Logging
 - Integriere Logging, um wichtige Ereignisse und Fehler zu protokollieren.
 - Erleichtert die Fehlerdiagnose und -behebung.

Best Practices Defensiver Programmierung

- Fallback-Mechanismen:
 - Immer vom schlimmsten Fall ausgehen
- Keine allgemeinen Exceptions abfangen
 - Allgemeine Exceptions können wichtige Fehlerinformationen verbergen und die Fehlersuche erschweren.
- Identischen (oder ähnlichen) Quellcode vermeiden
 - DRY ist pauschal nicht immer gültig
 - Zu viele Bedingungen für Sonderfälle können Komplexität und damit techn. Schuld nach oben treiben



Sicherstellung der Programmierrichtlinien

- Code Review
 - Reporting / Benachrichtigung bei CheckIn
- Automatische Benachrichtigung bei Verstößen
 - Team Foundation Server
- Checkin-Policies
- Überwachung der Stabilität und Fehlerrate

Benachrichtigung bei Verstößen

[Changeset 34760](#)

Summary

Team Project(s):	komsa.net
Checked in by:	[REDACTED]
Checked in on:	3/25/2009 2:03:54 PM
Code Reviewer:	None
Performance Reviewer:	None
Security Reviewer:	None
Comment:	None
Policy Override Reason:	Hotfix für [REDACTED]

Policy Failures

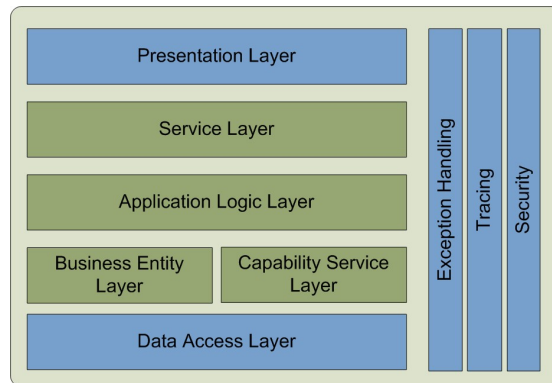
Type	Description
Changeset Comments Policy	Please provide some comments about your check-in
Code Analysis	One or more projects do not have an up-to-date build. Double-click this message for more information.
Work Items	You must associate this check-in with one or more work items.

Items

Name	Change	Folder
DataCache.cs	edit	\$/komsa.net/KomsaNetFramework/PersistenceFramework/Internal/Collections

Layer: Funktionalität logisch gruppieren

- Kontrolle von Abhängigkeiten durch Modularisierung





Continuous Integration

- Jeder Checkin triggert einen Build Prozess mit Unit Testing
- Verschiedenste Werkzeuge:
 - Team Foundation Server
 - TeamCity
 - CruiseControl .NET

Dokumentation

- Summary
- Summary Kommentare können automatisch mit GhostDoc erstellt werden
- ~~Sandcastle erlaubt das erzeugen von CHM Dateien aus den Summary Kommentaren~~
- DocFX

Lernings über Codequalität

- Softwarequalität ist programmiersprachen-unabhängig
- Programmieren in einer OO-fähigen Sprache ist noch nicht automatisch OOP/OOD
- Softwareentwicklung ist ganzheitlich, Programmierung (nur) ein Teil davon
- Hohe Qualität ist eine Voraussetzung für mittel- und langfristigen Erfolg
- Qualität erfordert Anfangsinvestition, viel Wissen und langen Atem

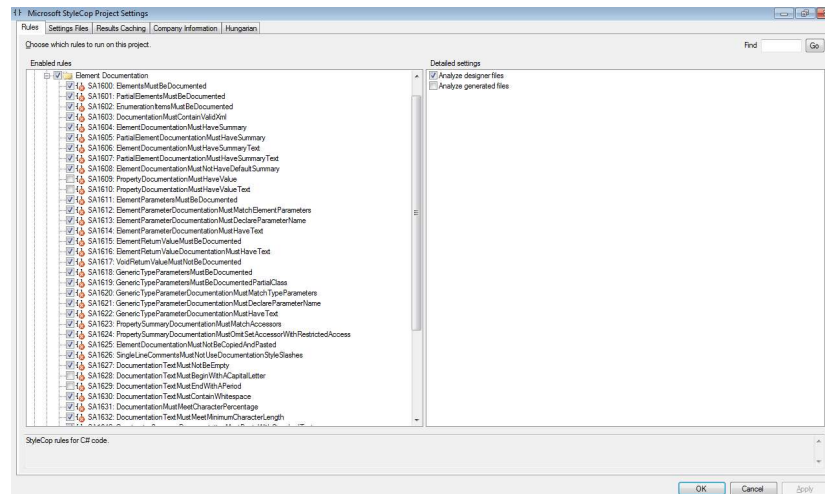
Was ist der StyleCop?

- Open Source Tool von Microsoft
- Soll den Entwickler(team) bei der Einhaltung von Codingstandards unterstützen
- Lässt sich in den ReSharper integrieren

Integration in den ReSharper

- Stylecop kann in den ReSharper mit einem Plugin integriert werden
 - StyleCop for ReSharper
- Erlaubt Syntax Highlighting und automatischen anpassen von Code nach den StyleCop regeln
- Regeln können angepasst werden!

Einstellungen des StyleCops



Warnings als Fehleranzeigen

- Bearbeiten der .csproj und StyleCop Targets einfügen
- Folgenden Eintrag verändern:
 - *StyleCopTreatErrorsAsWarnings auf false!*
- Darauf achten das die Version 4.4 und MSBuild Integration installiert ist.